

*Answer key — finished scripts and challenge solutions.*

**Welcome to Scratch C.** So far the cat could move, react, and choose - but it could not *remember* anything. Now it can. A *variable* is a named box that holds a number (or a word), like `score` or `lives`; you `set` it and `change` it, and its value shows live on the stage. Paired with green *Operators* - `pick random`, `+`, and the comparisons `<`, `=`, `>` - variables turn your projects into real games that keep score. Open `scratch.mit.edu` and click *Create*.

**The block groups you will use.** Each block has a colour. The colour tells you which group it is in. Find the colour first, then find the block.

|                  |               |                                                                                    |
|------------------|---------------|------------------------------------------------------------------------------------|
| <b>Motion</b>    | (blue)        | Move the sprite: move, turn, go to, point towards, change x or y.                  |
| <b>Looks</b>     | (purple)      | Change how the sprite looks: say, costume, size, backdrop.                         |
| <b>Sound</b>     | (pink)        | Play a sound.                                                                      |
| <b>Events</b>    | (yellow)      | Start a script (green flag, key, click, when I receive) and broadcast messages.    |
| <b>Control</b>   | (orange)      | Choose when and how often blocks run: wait, forever, repeat, if, stop.             |
| <b>Sensing</b>   | (light blue)  | Ask about the world: touching?, key pressed?, mouse, ask and answer.               |
| <b>Operators</b> | (green)       | Do maths and comparisons: plus, pick random, less-than, equals, more-than, and/or. |
| <b>Variables</b> | (dark orange) | Remember a value like score: set, change, and show a variable.                     |

## Project 1: A counter that remembers

*Make your first variable. Every time you click the cat, a number called `clicks` goes up by one, and the cat tells you the total. The variable remembers the count between clicks - that is the whole point.*

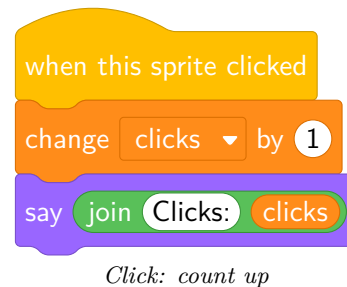
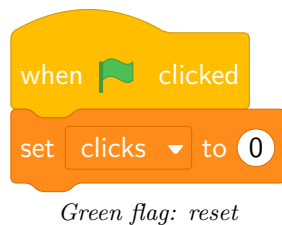
**Get ready.** Click the dark-orange *Variables* group, then *Make a Variable*. Name it `clicks` and click OK. A little box appears on the stage showing its value.

**New blocks.**

|                                    |                                                                                        |
|------------------------------------|----------------------------------------------------------------------------------------|
| <code>set [clicks] to ()</code>    | Put a number into the variable. Use this to start the count at 0.                      |
| <code>change [clicks] by ()</code> | Add to the variable. Change by 1 counts up; change by -1 counts down.                  |
| <code>(clicks)</code>              | The rounded reporter - drop it into any slot to use the value the box is holding.      |
| <code>join () ()</code>            | A green Operator that sticks two things together, like <i>Clicks</i> : and the number. |

**Build it.**

1. Make a `when green flag clicked` script with one block: `set [clicks] to (0)`. This resets the count each time you start.
  2. Make a second script: `when this sprite clicked`.
  3. Under it add `change [clicks] by (1)`.
  4. Then add `say (join (Clicks: ) (clicks))`. Drag the `join` Operator into the say slot, type *Clicks:* in its left box, and drop the `clicks` reporter into its right box.
  5. Click the green flag, then click the cat a few times. The number climbs and the cat reads it out.
- Two scripts: reset on the flag (left), count on each click (right):



The box on the stage and the `clicks` reporter are the same variable. Change it in one place and both update - the program is remembering one number for you.

**Your task.** Now make your project better. Do these in order. Each one builds on the last.

1. Add a reset key: make a `when [r] key pressed` script with `set [clicks] to (0)`.  
*Expected: Pressing r sets the count back to zero.*
2. Celebrate a milestone: in the click script, after the change, add `if <(clicks) = (10)> then say (Ten! You win.) for (2) seconds`.  
*Expected: At exactly 10 clicks the cat shouts Ten! You win.*
3. Grow with the count: also add `change size by (5)` in the click script so the cat gets bigger every click.  
*Expected: The cat grows a little each time, alongside the rising number.*

## Project 2: Whack-a-cat

The cat pops up at a random spot, waits a moment, then jumps somewhere new. Click it before it moves to score a point. A `score` variable keeps your total, and `pick random` makes every round different.

**Get ready.** Make a variable called `score` (Variables group, Make a Variable).

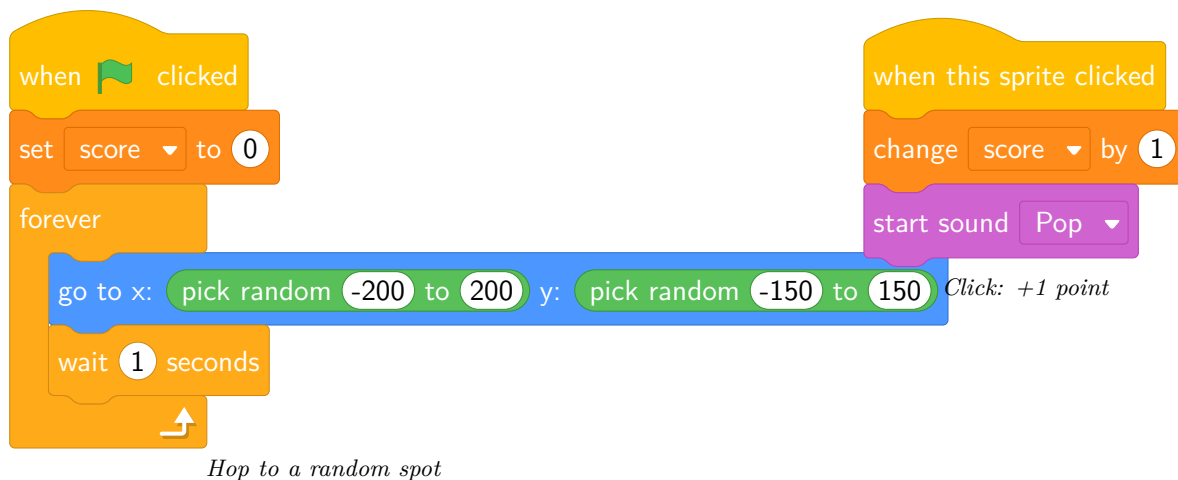
**New blocks.**

|                                   |                                                                                  |
|-----------------------------------|----------------------------------------------------------------------------------|
| <code>pick random () to ()</code> | A green Operator. It gives a fresh random number in the range each time it runs. |
| <code>go to x: () y: ()</code>    | Jump to a spot - here the x and y are random, so the cat appears anywhere.       |

**Build it.**

1. Start with `when green flag clicked`, then `set [score] to (0)`.
2. Add a `forever` loop.
3. Inside it, add `go to x: (pick random (-200) to (200)) y: (pick random (-150) to (150))`. Drop a `pick random` Operator into each of the x and y slots.
4. Under that, still inside the loop, add `wait (1) seconds`.
5. Make a second script: `when this sprite clicked`, then `change [score] by (1)` and `start sound [Pop]`.
6. Green flag. The cat hops around - click it quickly to run up the score.

The hopping loop (left) and the score-on-click script (right):



`pick random (1) to (6)` is like rolling a die: a new number every time. That is what keeps the game from being the same twice.

**Your task.** Now make your project better. Do these in order. Each one builds on the last.

1. Make it harder: change `wait (1) seconds` to `wait (0.6) seconds` so the cat does not sit still as long.  
*Expected: The cat moves sooner, leaving less time to click.*
2. Read the score aloud: in the click script add `say (join (Score: ) (score)) for (0.5) seconds`.  
*Expected: The cat announces the running score on each hit.*
3. Vary the timing: change `wait (1) seconds` to `wait (pick random (0.4) to (1.2)) seconds` so the pause is unpredictable.  
*Expected: Each hop waits a random amount, so the rhythm keeps changing.*

### Project 3: Guess my number

The computer thinks of a secret number from 1 to 10 and you try to guess it. After each guess it says higher or lower, and it keeps asking until you are right. This is a loop with a finish line: `repeat until` the guess matches.

**Get ready.** Make a variable called `secret`.

**New blocks.**

|                     |                                                                                           |
|---------------------|-------------------------------------------------------------------------------------------|
| ask () and wait     | A Sensing block. It shows a question and a text box, and waits for the player to type.    |
| (answer)            | A Sensing reporter holding whatever the player just typed.                                |
| repeat until <>     | A Control loop. It runs its blocks over and over until the hexagon question becomes true. |
| () > () and () = () | Green Operator comparisons. Each is true or false, so each fits an if or a loop.          |

### Build it.

1. Start with `when green flag clicked`.
2. Add `set [secret] to (pick random (1) to (10))` so the computer picks a hidden number.
3. Add `ask (I picked 1-10. Guess?) and wait`.
4. Drag in `repeat until <(answer) = (secret)>`. Build the hexagon with the green `() = ()` Operator, the `answer` reporter, and the `secret` reporter.
5. Inside the loop add an `if <> then ... else ...`. Put `(answer) > (secret)` in its hexagon, `say (Too big!) for (1) seconds` in the if part, and `say (Too small!) for (1) seconds` in the else part. Then add `ask (Guess again) and wait` so the player tries again.
6. After the loop add `say (You got it!) for (2) seconds`.
7. Green flag, and play. The cat guides you in until you land on the secret.

When you are done, your code should look like this:



*repeat until* checks its question before each pass. The moment your guess equals the secret, the loop stops and the cat congratulates you.

**Your task.** Now make your project better. Do these in order. Each one builds on the last.

1. Count the guesses: make a variable `guesses`, `set [guesses] to (0)` after the flag, and `change [guesses] by (1)` inside the loop. End with `say (join (Guesses: ) (guesses)) for (2) seconds`.  
*Expected: The cat reports how many tries it took.*
  2. Make it harder: change `pick random (1) to (10)` to `(1) to (100)` and update the question text.  
*Expected: A wider range means more guesses to home in.*
  3. Reveal the answer: change the ending to `say (join (It was ) (secret)) for (2) seconds`.  
*Expected: The cat shows the secret number it had chosen.*
- 

## Project 4: Mini project: catch the apple

A real score game. An apple falls from the top at a random spot; steer the cat to catch it and your `score` goes up. Miss it and it starts again from the top. Everything from this sheet, working together.

**Get ready.** Steer the cat with the four arrow-key scripts from *Scratch A* (Project 2). Then add an **Apple** sprite: click *Choose a Sprite* (bottom right) and pick Apple. Make a variable `score` *For all sprites*. The script below goes on the **Apple**.

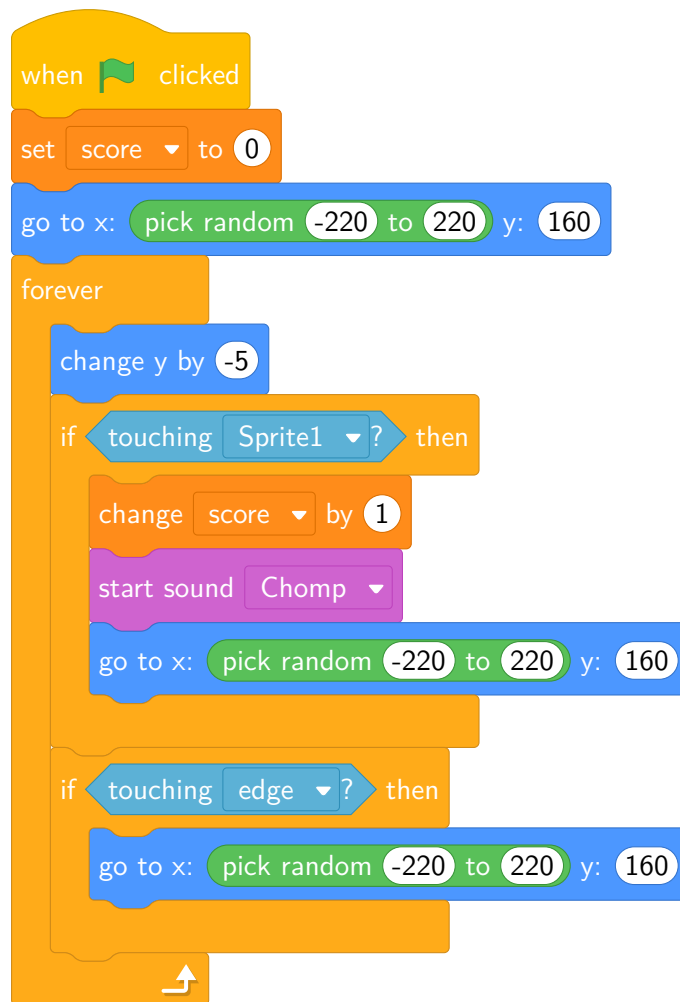
**New blocks.**

|                                  |                                                                                           |
|----------------------------------|-------------------------------------------------------------------------------------------|
| <code>touching [Sprite1]?</code> | A Sensing hexagon, true when the apple touches the cat (the cat sprite is named Sprite1). |
| <code>change y by ()</code>      | Move up or down. A negative number makes the apple fall.                                  |

**Build it.**

1. On the Apple, start with `when green flag clicked, then set [score] to (0)`.
2. Add `go to x: (pick random (-220) to (220)) y: (160)` so the apple starts at the top in a random column.
3. Add a `forever` loop. Inside, add `change y by (-5)` so the apple falls.
4. Still in the loop, add `if <touching [Sprite1]?> then change [score] by (1), start sound [Chomp], and go to x: (pick random (-220) to (220)) y: (160)` - caught, so score and respawn at the top.
5. Add one more `if <touching [edge]?> then go to x: (pick random (-220) to (220)) y: (160)` so a missed apple restarts from the top when it hits the floor.
6. Green flag. Catch falling apples with the arrow keys and watch the score climb.

The Apple's script (the cat uses the Scratch A arrow keys):



The apple only moves up and down within the random columns, so the only edge it ever reaches is the floor - that is how `touching [edge]?` spots a miss.

**Your task.** Now make your project better. Do these in order. Each one builds on the last.

1. Add lives: make a variable `lives`, `set [lives] to (3)` after the flag. In the `touching [edge]?` part add `change [lives] by (-1)`, then `if <(lives) = (0)>` then `say (Game Over) for (2) seconds` and `stop [all]`.

*Expected: A missed apple costs a life; at zero lives the game stops.*

2. Speed it up: change `change y by (-5)` to `change y by (-8)` so the apples fall faster.

*Expected: Faster apples are harder to catch.*

3. Add a second apple: right-click the Apple sprite and choose *duplicate*. The copy runs the same script on its own.

*Expected: Two apples fall at once, doubling the action.*